



Documentação técnica e metodológica

Cálculo de emissões de CO₂e do transporte de participantes, metodologia, fatores por tipo de deslocamento, limitações, operação offline e deploy.

Versão dos fatores de emissão: 1.0.0-2026.07

Documento gerado em 07 de julho de 2026.

Sumário

1. Metodologia de cálculo
 2. Cálculo de emissão — detalhamento por tipo
 3. Limitações das estimativas
 4. Funcionamento offline
 5. Instalação
 6. Como hospedar em produção
-

1. Metodologia de cálculo

Metodologia de cálculo

Escopo

O sistema estima **apenas** as emissões de CO₂ equivalente (CO₂e) do **deslocamento dos participantes até o evento**. Nenhuma outra fonte (energia, alimentação, resíduos, hospedagem) é considerada.

A metodologia é **inspirada na calculadora pública da Fundação SOS Mata Atlântica** (<https://www.sosma.org.br/calculadora-sua-emissao-de-co2>), que trabalha com estimativas de CO₂e para transporte a partir de referências de inventários de gases de efeito estufa (GHG Protocol Brasil / IPCC). O sistema **não consulta** o site da SOS Mata Atlântica em momento algum: não há scraping, não há chamada em tempo real. Os fatores vivem numa base **local e versionada**.

Fórmulas

```
emissão_kg_co2e = distância_km × fator_emissão (kg CO2e/km)
```

```
ida e volta:          distância_total = distância_ida × 2
```

```
veículo compartilhado: emissão_individual = emissão_do_veículo ÷ nº de pessoas no veículo
```

```
bicicleta / a pé:    emissão_kg_co2e = 0
```

Cada fator tem uma **base** (`factor_basis`):

- `vehicle` — fator por veículo (carro, moto, van, app/táxi): a emissão do veículo é dividida pelo número de ocupantes informado;
- `passenger` — fator já por passageiro-km (ônibus, avião, "outro"): **não** se divide por ocupantes.

Versionamento dos fatores

- Os fatores ficam em `backend/seeds/emission_factors.json` (fonte da verdade, com fonte documentada por fator) e são carregados na tabela `emission_factors` pelo seed. Uma cópia embarcada no totem (`totem/src/emissions/factors.json`) permite o cálculo 100% offline.
- Cada conjunto tem uma versão (atual: `1.0.0-2026.07`). Toda resposta grava em `transport_answers.calculation_version` a versão usada — os números de qualquer relatório são sempre rastreáveis ao conjunto de fatores da época.
- Ao atualizar fatores: crie uma **nova versão** no JSON do seed (nunca edite valores da versão antiga), rode `npm run seed` (desativa as versões anteriores e ativa a nova) e atualize a cópia do totem + rebuild.
- O servidor **recalcula** cada resposta recebida com os fatores ativos; se o valor do cliente divergir (>0,5 kg), grava o valor do servidor e registra `recalculated` em `sync_logs` .

Fatores da versão 1.0.0-2026.07 (kg CO₂e/km)

Modal	Combustível	Fator	Base	Derivação/fonte
Carro	Gasolina	0,187	veículo	2,212 kg CO ₂ e/L (GHG Protocol Brasil/MCTI) ÷ 11,8 km/L
Carro	Etanol	0,036	veículo	CO ₂ biogênico neutro; CH ₄ /N ₂ O residuais ≈0,436 kg CO ₂ e/L ÷ 8,1 km/L
Carro	Diesel	0,201	veículo	2,613 kg CO ₂ e/L (diesel B) ÷ 13 km/L
Carro	GNV	0,152	veículo	2,03 kg CO ₂ e/m ³ ÷ 13,4 km/m ³
Carro	Flex	0,126	veículo	média ponderada gasolina/etanol 60/40 (perfil ANP)
Carro	Outro	0,187	veículo	assume gasolina (conservador)
Moto	Gasolina	0,071	veículo	2,212 kg CO ₂ e/L ÷ 31 km/L
Moto	Etanol	0,016	veículo	análogo ao carro etanol, 27 km/L
Moto	Flex	0,049	veículo	média 60/40
Van	Diesel	0,278	veículo	2,613 kg CO ₂ e/L ÷ 9,4 km/L
Van	Gasolina	0,295	veículo	2,212 kg CO ₂ e/L ÷ 7,5 km/L
App/Táxi	(idem carro)	—	veículo	mesmos fatores do carro por combustível
Ônibus	—	0,027	passageiro	DEFRA UK 2023, categoria <i>coach</i> (rodoviário/fretado)
Avião	—	0,158	passageiro	DEFRA UK 2023, voo doméstico, sem forçamento radiativo
Bicicleta/A pé	—	0	passageiro	deslocamento ativo
Outro	—	0,100	passageiro	valor médio conservador entre modais motorizados (estimativa documentada)

Fontes primárias: Programa Brasileiro GHG Protocol (FGV/MCTI) para fatores de combustíveis; DEFRA UK *GHG Conversion Factors 2023* para fatores por passageiro-km; consumos médios de frota assumidos e documentados acima.

Conversão em árvores (somente no dashboard)

árvores = teto(emissão_total_kg ÷ 163,14)

163,14 kg CO₂ é a absorção média estimada de **uma árvore nativa da Mata Atlântica ao longo de ~20 anos**, valor usual em projetos brasileiros de restauração/neutralização (referência difundida por Iniciativa Verde/SOS Mata Atlântica). Essa conversão **nunca** aparece para o participante — apenas no dashboard administrativo e no relatório.

Texto metodológico dos relatórios

"Os cálculos apresentados são estimativas de emissões de CO₂ equivalente relacionadas exclusivamente ao transporte dos participantes até o evento. A metodologia foi inspirada na calculadora pública da Fundação SOS Mata Atlântica e utiliza fatores de emissão versionados, documentados e armazenados no sistema."

2. Cálculo de emissão — detalhamento por tipo

Este documento descreve **exatamente** a conta feita pelo sistema para cada tipo de deslocamento, com os fatores da versão **1.0.0-2026.07** e exemplos numéricos passo a passo.

O motor de cálculo é o mesmo no totem (offline, [totem/src/emissions/calc.js](#)) e no servidor ([backend/src/services/emissions.js](#)). O servidor **recalcula** toda resposta recebida e o valor dele prevalece.

1. Fórmula geral

Toda emissão individual sai da mesma base:

```
distância_total = round_trip ? distância_km × 2 : distância_km
emissão_bruta_kg = distância_total × fator_kg_co2e_por_km
```

Depois, **conforme a base do fator** (`factor_basis`), aplica-se ou não a divisão por ocupantes:

```
factor_basis = "vehicle" → emissão = emissão_bruta ÷ nº_de_ocupantes
factor_basis = "passenger" → emissão = emissão_bruta (sem divisão)
```

O resultado final é arredondado para **4 casas decimais**:

```
emissão_final = round(emissão × 10000) / 10000
```

Referência no código: [calc.js:34-51](#) e [emissions.js:39-53](#).

2. Tipos com fator por veículo (`vehicle`)

Aplicáveis a: **carro, moto, van, aplicativo/táxi**. O participante informa o **combustível** e o **nº de ocupantes**; a emissão do veículo é **rateada entre todos os ocupantes**.

```
emissão_individual = (distância_total × fator_do_combustível) ÷ nº_de_ocupantes
```

Se o combustível não for informado (ou não existir na base para aquele modal), usa-se o fator `"outro"` do próprio modal como fallback conservador ([emissions.js:27-31](#)).

Fatores (kg CO₂e/km)

Modal	Combustível	Fator	Derivação
Carro	Gasolina	0,187	2,212 kg CO ₂ e/L ÷ 11,8 km/L
Carro	Etanol	0,036	CO ₂ biogênico neutro; CH ₄ /N ₂ O residuais ≈ 0,436 kg CO ₂ e/L ÷ 8,1 km/L
Carro	Diesel	0,201	2,613 kg CO ₂ e/L ÷ 13 km/L
Carro	GNV	0,152	2,03 kg CO ₂ e/m ³ ÷ 13,4 km/m ³
Carro	Flex	0,126	média ponderada gasolina/etanol 60/40 (perfil ANP)
Carro	Outro	0,187	assume gasolina (conservador)
Moto	Gasolina	0,071	2,212 kg CO ₂ e/L ÷ 31 km/L
Moto	Etanol	0,016	análogo ao carro etanol, 27 km/L
Moto	Flex	0,049	média 60/40
Moto	Outro	0,071	assume gasolina
Van	Diesel	0,278	2,613 kg CO ₂ e/L ÷ 9,4 km/L
Van	Gasolina	0,295	2,212 kg CO ₂ e/L ÷ 7,5 km/L
Van	Outro	0,278	assume diesel (predominante em vans)
App/Táxi	Gasolina	0,187	mesmo fator do carro a gasolina
App/Táxi	Etanol	0,036	mesmo fator do carro a etanol
App/Táxi	GNV	0,152	mesmo fator do carro a GNV
App/Táxi	Flex	0,126	mesmo fator do carro flex
App/Táxi	Outro	0,187	assume gasolina

Exemplo A — Carro a gasolina, ida e volta, 3 ocupantes

Participante mora a **20 km** do evento, vai de **carro a gasolina**, marca **ida e volta** e informa **3 pessoas** no carro.

```

distância_total = 20 × 2           = 40 km
emissão_bruta   = 40 × 0,187       = 7,48 kg CO2e   (emissão do carro)
emissão_indiv.  = 7,48 ÷ 3         ≈ 2,4933 kg CO2e   (por pessoa)

```

Resultado registrado para esse participante: 2,4933 kg CO₂e.

Exemplo B — Moto a gasolina, só ida, 1 ocupante

Mora a **15 km**, vai de **moto, só ida**, sozinho.

```

distância_total = 15           = 15 km
emissão_bruta   = 15 × 0,071   = 1,065 kg CO2e
emissão_indiv.  = 1,065 ÷ 1    = 1,065 kg CO2e

```

Resultado: 1,065 kg CO₂e.

3. Tipos com fator por passageiro (`passenger`)

Aplicáveis a: **ônibus, avião, outro**. O fator já é expresso **por passageiro-km**, então **não** há divisão por ocupantes nem escolha de combustível.

$$\text{emissão_individual} = \text{distância_total} \times \text{fator_por_passageiro_km}$$

Fatores (kg CO₂e/km)

Modal	Fator	Base	Fonte
Ônibus	0,027	passageiro	DEFRA UK 2023, categoria <i>coach</i> (rodoviário/fretado)
Avião	0,158	passageiro	DEFRA UK 2023, voo doméstico, sem forçamento radiativo
Outro	0,100	passageiro	valor médio conservador entre modais motorizados

Exemplo C — Ônibus, ida e volta, 120 km por trecho

$$\begin{aligned} \text{distância_total} &= 120 \times 2 &&= 240 \text{ km} \\ \text{emissão} &= 240 \times 0,027 &&= 6,48 \text{ kg CO}_2\text{e} \end{aligned}$$

Resultado: 6,48 kg CO₂e (o nº de pessoas no ônibus é irrelevante — o fator já é por passageiro).

Exemplo D — Avião, só ida, 500 km

$$\begin{aligned} \text{distância_total} &= 500 &&= 500 \text{ km} \\ \text{emissão} &= 500 \times 0,158 &&= 79,0 \text{ kg CO}_2\text{e} \end{aligned}$$

Resultado: 79,0 kg CO₂e.

4. Tipo com emissão zero

Bicicleta / A pé — deslocamento ativo, emissão direta considerada **zero**. O cálculo faz *short-circuit* antes de qualquer conta ([calc.js:35](#)):

$$\text{emissão} = 0 \quad (\text{independente de distância, ida/volta ou ocupantes})$$

5. Recálculo e validação no servidor

Ao sincronizar, o servidor **refaz a conta** com os fatores ativos e grava o valor dele. Se o valor enviado pelo cliente divergir em **mais de 0,5 kg**, a resposta é marcada como `recalculated` em `sync_logs` (`sync.js:87-109`):

```
mismatch = | emissão_servidor - emissão_cliente | > 0,5 kg
```

O valor persistido em `transport_answers.emission_kg_co2e` é **sempre o do servidor**, junto com a `calculation_version` usada — garantindo rastreabilidade histórica dos números.

6. Agregações do dashboard / relatório

Sobre o conjunto de respostas de um evento (`admin.js:110-134`):

```
total_co2e    = Σ emissão de todas as respostas
avg_co2e      = média das emissões por participante
por_modal     = Σ emissão agrupada por transport_mode
% do modal    = co2e_do_modal ÷ total_co2e × 100
```

Conversão em árvores (só no dashboard, nunca exibida ao participante)

```
árvores = teto( total_co2e ÷ 163,14 )
```

163,14 kg CO₂ é a absorção média estimada de **uma árvore nativa da Mata Atlântica ao longo de ~20 anos**. Usa-se `Math.ceil` (arredonda **para cima**), pois neutralizar exige plantar a árvore inteira (`admin.js:133`).

Exemplo E — Evento com 3 participantes

Somando os exemplos A (2,4933), C (6,48) e um terceiro qualquer que totalize **10,075 kg CO₂e** no evento:

```
total_co2e = 10,075 kg CO2e
árvores    = teto(10,075 ÷ 163,14) = teto(0,0618) = 1 árvore
```

7. Resumo — qual conta cada tipo usa

Tipo	Escolhe combustível?	Divide por ocupantes?	Fórmula
Carro	✓	✓	$\text{dist} \times \text{fator_combustível} \div \text{ocupantes}$
Moto	✓	✓	$\text{dist} \times \text{fator_combustível} \div \text{ocupantes}$
Van	✓	✓	$\text{dist} \times \text{fator_combustível} \div \text{ocupantes}$
App/Táxi	✓	✓	$\text{dist} \times \text{fator_combustível} \div \text{ocupantes}$
Ônibus	✗	✗	$\text{dist} \times 0,027$
Avião	✗	✗	$\text{dist} \times 0,158$
Outro	✗	✗	$\text{dist} \times 0,100$
Bicicleta / A pé	✗	✗	0

`dist` = distância já considerando ida e volta ($\times 2$ quando `round_trip`).

Fontes primárias dos fatores: Programa Brasileiro GHG Protocol (FGV/MCTI) para combustíveis; DEFRA UK *GHG Conversion Factors 2023* para fatores por passageiro-km. Metodologia inspirada na calculadora pública da Fundação SOS Mata Atlântica. Ver [docs/metodologia.md](#) e [backend/seeds/emission_factors.json](#) (fonte da verdade, com fonte documentada por fator).

3. Limitações das estimativas

Limitações das estimativas

Os números produzidos pelo sistema são **estimativas de ordem de grandeza**, adequadas para sensibilização e para dimensionar ações de compensação — não são um inventário auditável de gases de efeito estufa.

Limitações dos dados de entrada

- **Distância autodeclarada:** o participante informa um valor aproximado, sem verificação por rota real.
- **Ocupação do veículo autodeclarada:** o rateio por ocupante depende do número informado.
- **Um único modal por resposta:** trajetos multimodais (ex.: avião + app) são representados pelo modal principal escolhido pelo participante.
- **Sem deduplicação de pessoas:** quem responde duas vezes no totem gera dois registros (a deduplicação por UUID evita duplicar *sincronizações*, não *pessoas*).

Limitações dos fatores de emissão

- Fatores usam **consumos médios de frota** (km/L) assumidos e documentados — o veículo real do participante pode consumir mais ou menos.
- O fator **flex** assume um mix fixo gasolina/etanol (60/40).
- O fator de **ônibus** usa a categoria *coach* (rodoviário/fretado) com ocupação média internacional; ônibus urbanos lotados ou vazios divergem.
- O fator de **avião** é um valor médio de voos domésticos por passageiro-km, **sem** forçamento radiativo (incluí-lo poderia até dobrar o valor) e sem distinção de classe ou aeronave.
- O modal "**outro**" usa um valor médio conservador (0,100 kg CO₂e/km) por definição impreciso.
- **Aplicativo/táxi** não contabiliza os km rodados vazios até o embarque.
- Fatores de combustível são de escopo de **queima (tail-pipe)**; a cadeia de produção do combustível (well-to-tank) não está incluída.

Limitações da conversão em árvores

A conversão (163,14 kg CO₂/árvore em ~20 anos) é uma média usada em projetos de restauração da Mata Atlântica. A absorção real varia com espécie, solo, clima e mortalidade. O número de árvores é um **indicador de comunicação**, não um compromisso técnico de compensação — para neutralização formal, contrate um projeto certificado.

Recomendação de uso

Comunique sempre os resultados como "emissão **estimada**", mantenha a observação metodológica nos relatórios e preserve a `calculation_version` de cada resposta para rastreabilidade.

4. Funcionamento offline

Funcionamento offline

O totem é **offline-first**: depois de carregado uma vez com internet, todo o fluxo do participante funciona sem rede — inclusive o cálculo de CO₂e.

As três camadas

1. Service Worker — o app abre sem internet

O build do totem (via `vite-plugin-pwa` /Workbox) pré-cacheia **todos** os arquivos do app (HTML, JS, CSS, ícones). Ao abrir a URL, o Service Worker serve tudo do cache local, com ou sem rede. Atualizações do app são baixadas em segundo plano quando houver internet (`registerType: autoUpdate`).

2. IndexedDB (Dexie) — nada se perde

Cada resposta finalizada vira um registro na tabela `answers` do banco local `carbono-zero-totem` , com:

- `local_uuid` — UUID v4 gerado no dispositivo (chave de deduplicação global);
- payload completo (participante + transporte + emissão calculada + versão dos fatores);
- `status` : `pending` → `synced` .

O IndexedDB é persistente: fechar o navegador, recarregar a página ou cair a energia **não** apaga a fila. Além disso, o formulário em andamento é salvo em `localStorage` como rascunho — se a página recarregar no meio do preenchimento, o participante continua de onde parou.

3. Fila de sincronização — envio automático

A sincronização (`src/sync.js`) tenta enviar as respostas `pending` para `POST /api/sync/answers` :

- imediatamente após cada resposta;
- quando o navegador dispara o evento `online` ;
- a cada 60 segundos.

Regras:

- **Sucesso** → resposta marcada como `synced` (e apagada após 7 dias, por minimização de dados/LGPD).
- **Falha de rede/servidor** → nada muda; os dados continuam `pending` e serão reenviados na próxima tentativa. Não há limite de tentativas.
- **Duplicidade** → o servidor tem índice único em `local_uuid` ; reenvios retornam `duplicate` e o cliente marca como sincronizado. Enviar duas vezes nunca duplica dados.
- **Resposta inválida** (não deveria ocorrer com o app oficial) → o servidor registra o erro em `sync_logs` e o cliente remove da fila para não travar as demais.

Monitoramento

- **No totem:** a tela inicial mostra o estado da conexão e quantas respostas aguardam envio; a tela de configuração (7 toques no canto superior esquerdo) tem o botão "Sincronizar agora".
- **No dashboard:** a página **Sincronização** lista tudo que chegou ao servidor (`ok` , `duplicate` , `recalculated` , `error`) com mensagens de erro.

Cuidados operacionais

- Carregue o app **uma vez com internet** antes do evento (é isso que instala o Service Worker).
- Não use aba anônima/convidado: o IndexedDB é apagado ao fechar.
- Não limpe dados de navegação do dispositivo enquanto houver respostas pendentes.
- PWA exige **HTTPS** (ou `localhost` em desenvolvimento).

5. Instalação

Instalação

Requisitos

- Node.js 20 ou superior
- PostgreSQL 14+ (ou Docker, usando o `docker-compose.yml` da raiz)
- Navegador moderno (Chrome/Edge recomendados para o modo quiosque)

Desenvolvimento local

1. Banco de dados

```
docker compose up -d
```

Ou aponte `DATABASE_URL` para um PostgreSQL existente.

2. Backend

```
cd backend
cp .env.example .env          # ajuste DATABASE_URL, JWT_SECRET, CORS_ORIGINS
npm install
npm run migrate                # cria as tabelas (idempotente)
npm run seed                   # insere os fatores de emissão versionados
npm run create-admin admin@evento.com "SenhaForte123" "Nome do Admin"
npm run dev                    # http://localhost:3001
```

3. Totem

```
cd totem
npm install
npm run dev                    # http://localhost:5173
```

Em desenvolvimento o Service Worker não é gerado. Para validar o comportamento offline real:

```
npm run build
npm run preview                # http://localhost:4173
```

4. Dashboard

```
cd dashboard
cp .env.example .env          # VITE_API_URL=http://localhost:3001
npm install
npm run dev                    # http://localhost:5174
```

5. Configuração inicial

1. Faça login no dashboard e crie um evento em **Eventos** — anote o **ID**.
2. Abra o totem com `?config=1` (ou toque 7 vezes no canto superior esquerdo da tela inicial), informe o ID do evento, o nome exibido e a URL da API.
3. Pronto: o totem já grava respostas localmente e sincroniza quando houver rede.

Produção

Backend

- Hospede o Node.js (PM2, systemd, Docker etc.) atrás de um proxy HTTPS (Nginx/Caddy). **HTTPS é obrigatório**: Service Worker e PWA só funcionam em origem segura.
- Defina obrigatoriamente: `NODE_ENV=production`, `JWT_SECRET` forte, `DATABASE_URL` do banco gerenciado, `CORS_ORIGINS` com os domínios reais do totem e do dashboard (nunca `*`).
- Rode `npm run migrate` e `npm run seed` a cada deploy (ambos idempotentes).
- Faça backup regular do PostgreSQL — os dados pessoais coletados estão lá.

Totem e dashboard (estáticos)

```
cd totem && npm run build      # gera totem/dist
cd dashboard && npm run build   # gera dashboard/dist
```

- Sirva cada `dist/` como site estático (Nginx, Vercel, Netlify, S3+CDN...), cada um em seu domínio/subdomínio, sempre com HTTPS.
- No dashboard, defina `VITE_API_URL` **antes** do build.
- Ambos são SPAs: configure fallback de rotas para `index.html`.

Preparação do totem no evento

1. **Com internet**, abra a URL do totem uma vez no dispositivo — o Service Worker baixa e guarda todo o app.
2. Configure o evento (`?config=1`) e instale como app (menu "Instalar aplicativo") ou abra o navegador em modo quiosque: `chrome --kiosk https://totem.seudominio.com.br`
3. A partir daí o totem funciona **sem internet**. As respostas ficam na fila local e sincronizam sozinhas quando a rede voltar (ou leve o dispositivo a um local com internet após o evento e toque em "Sincronizar agora" na tela de configuração).
4. Antes de desligar/reinstalar o dispositivo, confirme na tela de configuração que há **0 respostas pendentes** — a fila vive no IndexedDB do navegador e é apagada se o perfil/dados do navegador

forem limpos.

6. Como hospedar em produção

Como hospedar em produção (passo a passo)

O sistema tem **3 partes** para colocar no ar, mais um **banco de dados**:

Parte	O que é	Onde hospedar (sugestão gratuita/barata)
Banco	PostgreSQL	Neon (grátis)
API (<code>backend/</code>)	Servidor Node	Render ou Railway
Dashboard (<code>dashboard/</code>)	Site estático	Vercel ou Netlify
Totem (<code>totem/</code>)	Site estático / PWA	Vercel ou Netlify

Regra de ouro: tudo precisa estar em **HTTPS**. O totem só funciona offline (Service Worker/PWA) em endereço seguro — Vercel, Netlify e Render já dão HTTPS automático.

Antes de tudo, suba o código para um repositório no **GitHub** (as três pastas). Todas as plataformas abaixo fazem deploy direto do GitHub.

1. Banco de dados — Neon (grátis)

1. Crie conta em <https://neon.tech>.
2. Crie um projeto → ele te dá uma **connection string** parecida com: `postgres://usuario:senha@ep-xxx.neon.tech/neondb?sslmode=require`
3. Guarde essa string — é o seu `DATABASE_URL`.

(Alternativas equivalentes: Supabase, Railway Postgres, ElephantSQL.)

2. API — Render (grátis para testar)

1. Crie conta em <https://render.com> e conecte seu GitHub.
2. **New** → **Web Service** → escolha o repositório → **Root Directory:** `backend`.
3. Render detecta o `Dockerfile` que já existe na pasta. (Se preferir sem Docker: Build `npm install`, Start `npm run migrate && npm run seed && npm start .`)
4. Em **Environment**, adicione as variáveis:

Variável	Valor
<code>DATABASE_URL</code>	a connection string da Neon
<code>JWT_SECRET</code>	uma senha longa e aleatória (invente 40+ caracteres)
<code>JWT_EXPIRES_IN</code>	<code>8h</code>
<code>NODE_ENV</code>	<code>production</code>
<code>CORS_ORIGINS</code>	as URLs do dashboard e do totem (preencher no passo 5)

5. Faça o deploy. Render te dá uma URL tipo `https://carbono-zero-api.onrender.com` — essa é a **URL da API**.

6. Crie o primeiro admin. No painel do Render, aba **Shell**, rode:

```
npm run create-admin seu@email.com "SuaSenhaForte" "Seu Nome"
```

(O `migrate` e o `seed` já rodaram sozinhos no boot, pelo Dockerfile.)

Atenção ao plano grátis do Render: ele "dorme" após 15 min sem uso e demora ~30s para acordar. Para um evento de verdade, use o plano **Starter (~US\$7/mês)** ou o **Railway**, que não dormem. O totem não perde nada se a API dormir — as respostas ficam na fila local e sincronizam quando ela acordar.

3. Dashboard — Vercel

1. Crie conta em <https://vercel.com> e conecte o GitHub.
2. **Add New** → **Project** → escolha o repositório → **Root Directory:** `dashboard`.
3. Framework: Vite. Em **Environment Variables**, adicione:

Variável	Valor
<code>VITE_API_URL</code>	a URL da API do passo 2 (ex.: <code>https://carbono-zero-api.onrender.com</code>)

4. Deploy. Você recebe uma URL tipo `https://carbono-zero-dashboard.vercel.app`.

A `VITE_API_URL` é lida **no momento do build**. Se mudar a URL da API depois, faça um novo deploy (Redeploy) do dashboard.

4. Totem — Vercel (outro projeto)

1. **Add New** → **Project** de novo → mesmo repositório → **Root Directory:** `totem`.

2. Não precisa de variável de ambiente: o totem descobre a API na tela de configuração (você digita a URL lá dentro no dia do evento).
3. Deploy. Você recebe uma URL tipo `https://carbono-zero-totem.vercel.app`.

5. Fechar o CORS

Volte no Render (passo 2), edite `CORS_ORIGINS` e coloque as duas URLs finais, separadas por vírgula, **sem / no fim**:

```
CORS_ORIGINS=https://carbono-zero-dashboard.vercel.app,https://carbono-zero-totem.vercel.app
```

Salve — o Render redeploys sozinho. Sem isso, o navegador bloqueia as chamadas.

6. Usar no evento

1. Abra o **dashboard**, faça login, crie o evento e anote o **ID**.
2. Abra o **totem** com `?config=1` (ex.: `https://carbono-zero-totem.vercel.app/?config=1`), informe o ID do evento e a URL da API. Toque em "Salvar e iniciar".
3. **Instale o totem no tablet** (menu do navegador → "Instalar app" / "Adicionar à tela inicial") ou abra em modo quiosque. Faça isso **uma vez com internet** para o Service Worker baixar o app — depois ele funciona offline.

Sobre o "APK" (importante)

O totem é um **PWA**, não um app nativo — por isso ele **não precisa de APK**. No tablet você usa "Adicionar à tela inicial" e ele vira um ícone/app em tela cheia, funcionando offline. Para totem de evento, esse é o caminho recomendado (nada de loja de apps, nada de instalar arquivo).

Se você realmente quiser um arquivo `.apk` (para a Play Store ou para distribuir manualmente):

1. Hospede o totem primeiro (passo 4 acima).
2. Vá em <https://www.pwabuilder.com>, cole a URL do totem.
3. Ele valida o PWA e gera um **pacote Android (`.apk` / `.aab`)** pronto — é a forma oficial do Google (usa Trusted Web Activity por baixo).
4. Baixe o APK e instale no tablet, ou publique na Play Store com o `.aab`.

Ou seja: o APK, se você quiser, é gerado **a partir** do site do totem já hospedado — não é um build separado do projeto.

Resumão do que vai existir no fim

- **Banco:** Neon (privado, só a API acessa)
- **API:** <https://...onrender.com> — recebe sincronização e serve o dashboard
- **Dashboard:** <https://...vercel.app> — login da equipe
- **Totem:** <https://...vercel.app> — PWA instalável (ou vira APK via PWABuilder)